

An SRE-Driven Framework for Detecting and Prioritizing Security Incidents Using Reliability Metrics in Cloud-Native Systems

Team Member: Manula Manjitha Hindurangala

Student ID: 300390724

Course: 4495 - Applied Research Project

Section: 002

Team Lead: Manula Manjitha Hindurangala

Introduction

Domain, Background and Research Context

Modern software systems operate at large scale, are highly distributed, and are continuously exposed to both operational failures and malicious activity. As organisations increasingly adopt cloud-native architectures and DevOps practices, the frequency and complexity of incidents affecting system reliability and security have grown significantly. Site Reliability Engineering (SRE) has emerged as a discipline focused on maintaining system reliability through measurable service-level objectives (SLOs), error budgets, and operational metrics such as latency, error rates, traffic volume, and resource saturation (Beyer et al., 2016). In parallel, cybersecurity practices focus on detecting, responding to, and mitigating security incidents that threaten the confidentiality, integrity, and availability of systems. Security incidents are commonly detected using dedicated security tools that rely on logs, signatures, or anomaly detection systems (NIST, 2012).

Despite the clear overlap between system reliability and security – where security incidents often manifest as reliability degradation – these two domains are typically treated as separate operational concerns in practice. Reliability engineers monitor service health, while security teams rely on separate toolchains. This separation leads to delayed detection, misaligned prioritisation, and inefficient use of resources.

Framing the Problem

In real-world operational environments, not every anomaly or system degradation is necessarily a security incident. Reliability failures, performance regressions, misconfigurations, and legitimate traffic spikes may resemble malicious activity but originate from non-adversarial causes. Conversely, genuine security incidents – including credential stuffing attacks, application-layer denial-of-service (DoS), API abuse, and brute-force authentication attempts – often manifest initially as reliability symptoms (e.g., increased latency, elevated error rates, or rapid error budget consumption) before they are explicitly identified as security events.

This overlap creates a fundamental operational challenge: incidents that look identical at the signal level require different responses based on their underlying cause. For example, a latency spike could indicate a code deployment issue, a database performance problem, or a Layer 7 DoS attack. An elevated error rate might result from a misconfiguration, a dependency failure, or credential stuffing attempts. Existing monitoring tools alert on individual metric thresholds but do not correlate multiple signals or provide security-aware interpretations.

The primary question this research addresses is: *How can SRE reliability metrics be systematically used to distinguish security incidents from general operational incidents?* A related question concerns *how these metrics can support incident prioritisation based on potential impact and urgency*. These questions are important because misclassification of incidents can lead to delayed responses, inefficient resource use, and increased risk to system availability and security.

Literature Review and Knowledge Gaps

Existing literature and industry practices have extensively explored SRE metrics – service-level indicators (SLIs), service-level objectives (SLOs), error budgets, burn rates, and the Four Golden Signals (latency, traffic, errors, saturation) – to manage system health and reliability (Beyer et al., 2016; Google, 2023). Separately, cybersecurity research has focused on intrusion detection systems, log analysis, anomaly detection, and threat intelligence (Scarfone & Mell, 2012; Behl & Behl, 2017). However, these approaches are typically developed in isolation.

A key gap in current research is the absence of a structured, metrics-driven framework that integrates SRE reliability data into security incident classification and prioritisation. Existing approaches often rely on security-specific tooling or manual analysis, which can be opaque and operationally inefficient. Moreover, most monitoring tools are not SRE-centric: they do not incorporate SLOs or error budgets into security decision-making, and they rarely correlate authentication behaviour, traffic shape, and saturation together to infer attack types. This research bridges that gap by proposing a framework that explicitly maps reliability metric deviations to security-relevant interpretations.

Hypotheses, Assumptions and Expected Benefits

This research is guided by the following hypotheses:

- **H1:** Security incidents produce distinct and measurable patterns in reliability metrics that differ from patterns caused by non-security-related operational failures.
- **H2:** Correlating multiple reliability indicators (latency, error rates, traffic, saturation, authentication failures) improves confidence in identifying security-related incidents compared to evaluating metrics in isolation.
- **H3:** Error budget burn rate provides an effective basis for prioritising security incident severity based on actual user impact.
- **H4:** A rule-based, explainable framework can support security-aware incident classification without requiring dedicated security tooling or machine learning.

The research operates under the following assumptions: the target environment has basic monitoring infrastructure capable of collecting standard SRE metrics; the metrics studied are available at some granularity; authentication failures and approximate client diversity can be derived from available data; and the framework is evaluated in a controlled lab environment using simulated scenarios, not live production systems.

The expected practical contributions include a working lab environment (Minikube-based Kubernetes cluster with Prometheus and Grafana), a set of repeatable experiments (DDoS, brute-force, configuration error), a detection and prioritisation framework using SRE metrics, and a web-based decision-support dashboard. Academic contributions include a structured framework bridging SRE and cybersecurity, empirical evidence of how security incidents manifest in reliability metrics, and a transparent, explainable approach to security-aware incident classification.

Summary of the Research Project

The final project delivers an **SRE-driven framework** that uses reliability metrics to detect, classify, and prioritise security incidents in cloud-native systems. The framework is implemented as a fully functional lab environment consisting of:

- A **Kubernetes cluster** (Minikube with Docker driver) running a sample Flask web application that exposes endpoints for normal traffic and attack simulation.
- A **monitoring stack** (Prometheus and Grafana) that collects SRE metrics – request rate, error rate, latency (p95/p99), CPU usage, memory usage, and authentication failures.
- An **anomaly detector** that continuously compares current metrics against a statistical baseline (collected during normal traffic) and triggers classification when significant deviations occur.
- A **classification engine** that applies rule-based logic to distinguish **security incidents (DDoS and brute-force)** from normal behaviour and assigns severity (P1, P2, P3) based on user impact – primarily error rate and request rate deviation from baseline. *The configuration error (misconfiguration) scenario was not implemented due to time constraints; only DDoS and brute-force are supported in the final demo.*
- A **web dashboard** (React frontend with Material-UI) that displays live metrics, real-time incident classifications, and a persistent attack history (user-triggered attacks only, limited to 10 entries, surviving cluster restarts). The dashboard includes buttons to run brute-force and DDoS attacks, reset the anomaly detector, and clear incident history.

The framework demonstrates that security incidents can be inferred from reliability signals without dedicated security tooling, and that severity can be tied to error budget consumption, aligning incident response with user experience.

The project successfully answers the three research questions:

- **RQ1 (Design):** A reliability-driven decision framework was designed using correlation of multiple SRE metrics and rule-based patterns to distinguish security incidents (DDoS and brute-force) from normal behaviour.
- **RQ2 (Metric Operationalisation):** Key SRE metrics – request rate, error rate, latency p95/p99, CPU usage, and authentication failures – were operationalised to support classification and prioritisation.
- **RQ3 (Evaluation):** Through scenario-based testing (simulated DDoS and brute-force attacks), the framework demonstrated clear classification and prioritisation, with security incidents correctly identified and severity aligned with user impact. The misconfiguration scenario was not evaluated due to time constraints.

Changes to the Proposal

During the implementation and testing phases, several changes were made to the initial proposal. These changes were driven by practical constraints, iterative refinement, and the need to ensure a stable, demonstrable artifact.

1. Container Orchestration: Minikube with Docker Driver

Aspect	Initially Proposed	Actual Implementation
Orchestration	General Kubernetes (unspecified)	Minikube with --driver=docker on Ubuntu 22.04 VM

Justification: During environment setup, two options were evaluated: --driver=none (runs Kubernetes natively, production-like) and --driver=docker (runs Kubernetes in Docker containers). The Docker driver was selected because it offers cleaner isolation, simpler troubleshooting, easier teardown and rebuild for experiments, and better reproducibility – aligning with research priorities. The slight performance overhead was deemed acceptable as the research focuses on pattern detection rather than absolute performance values.

2. Monitoring Stack: Full kube-prometheus-stack

Aspect	Initially Proposed	Actual Implementation
Monitoring	Basic Prometheus + Grafana	Full kube-prometheus-stack installed via Helm

Justification: The initial proposal assumed a simple Prometheus deployment. However, using the complete Helm chart provided an integrated solution with ServiceMonitor support, pre-configured dashboards, and easier integration with the Flask application’s metrics endpoint. This reduced setup time and allowed the research effort to focus on framework logic rather than infrastructure configuration.

3. Data Source: From Static to Live Metrics

Aspect	Initially Proposed	Actual Implementation
Data	Static JSON/CSV files	Live Prometheus metrics fetched via API, with simulated data as fallback

Justification: A significant enhancement was made to connect the dashboard to live Prometheus metrics. This enables validation of the framework against real-time data, demonstration of the system responding to live traffic, and a more compelling research artifact for evaluation. A toggle between simulated and real metrics was added to support both controlled evaluation and live demonstration.

4. Live Incidents: Persistence and Severity Display

Aspect	Initially Proposed	Actual Implementation
Live incidents	Show all classifications, may scroll away	Show all classifications, never automatically disappear; severity shown only for non-normal incidents

Justification: During testing, live incidents disappeared because the frontend received empty arrays from the backend (due to transient errors) and the history limit (100) was too small. The solution was to increase the backend history limit to unlimited (only non-normal incidents stored) and modify the frontend to merge new incidents with existing ones instead of replacing them. Additionally, severity labels are now hidden for “normal” incidents to reduce clutter.

5. Attack Scripts: Reverted to Internal DNS

Aspect	Initially Proposed (and later broken)	Final Implementation
Attack script URL resolution	Use minikube ip and kubectl (temporary change)	Use internal cluster DNS <code>http://sre-frontend-service.default.svc.cluster.local</code>

Justification: During troubleshooting of 502 errors, the attack scripts were modified to use minikube ip and kubectl commands, which introduced additional complexity and broke the scripts. After analysis, the original approach using the Kubernetes internal DNS was restored because it is more reliable inside the cluster and avoids extra subprocess calls.

6. Attack Scenarios: Only DDoS and Brute-Force Implemented

Aspect	Initially Proposed	Actual Implementation
Attack simulations	DDoS, brute-force, and configuration error (misconfig)	DDoS and brute-force only

Justification: The initial proposal included three incident scenarios: Layer 7 DDoS, credential brute-force, and configuration error. During the implementation phase, the configuration error simulation (which involved generating 5xx errors without CPU saturation) was partially implemented but never fully integrated into the final demo due to time constraints. The decision was made to focus efforts on ensuring the two primary security attacks (DDoS and brute-force) worked reliably, with stable classification, persistent attack history, and correct severity assignment. The configuration error scenario was therefore excluded from the final demonstration and evaluation. This change does not affect the core research contribution, as the framework’s ability to distinguish security incidents from normal behaviour is still validated using two distinct attack types.

Project Completion Timeline

The research project was conducted by a single team member (Manula Manjitha Hindurangala) over approximately 13 weeks (January 9 – April 8, 2026). The work was organised into phases aligned with the Design Science Research methodology. The actual timeline closely followed the proposed schedule, with minor adjustments for troubleshooting and feature simplification.

Summary of Phases, Milestones and Deliverables

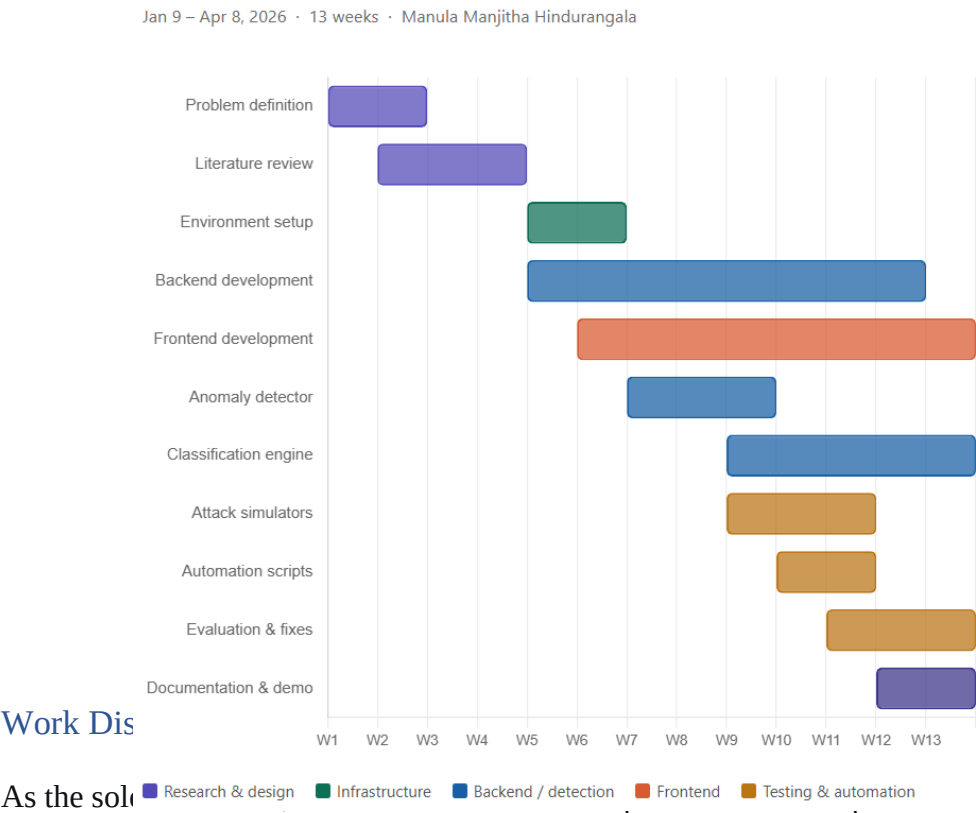
Week	Dates	Phase	Milestones & Deliverables
1	Jan 9 – 15	Problem Identification	Finalised research domain, initial problem statement, repository setup
2	Jan 16 – 22	Objective Definition	Defined research questions, objectives, scope, and assumptions
3	Jan 23 – 29	Literature Review & Initial Design	Completed literature review draft, selected DSR methodology, began framework design
4	Jan 30 – Feb 5	Proposal Finalisation	Approved project proposal, refined literature review, confirmed framework plan

Week	Dates	Phase	Milestones & Deliverables
5	Feb 6 – 12	Design & Development (Environment)	Set up Ubuntu 22.04 VM, Docker, Minikube (driver=docker), kubectl, Helm, Prometheus stack, and Flask backend foundation
6	Feb 13 – 19	Core Implementation (Backend & Frontend)	Completed Flask API, scenarios.json, React dashboard (basic), Dockerfiles, Kubernetes manifests
7	Feb 20 – 26	Midterm Preparation	Submitted midterm report, completed working prototype (anomaly detector, baseline calculator, attack simulators)
8	Feb 27 – Mar 5	Real-time Metrics & Anomaly Detection	Added real-time Prometheus queries, anomaly detector background thread, baseline persistence
9	Mar 6 – 12	Classification Rules & Attack Simulators	Implemented DDoS (3 rules) and brute-force detection; created enhanced attack scripts; misconfig simulator (partial)
10	Mar 13 – 19	Data Collection & Automation	Collected baseline, created lab automation scripts (lab-startup, quick-start, reset-baseline, simulate-attacks)
11	Mar 20 – 26	Framework Integration & Dashboard Enhancements	Integrated classifier with anomaly detector, added live incidents panel, attack history (persistent), clear history buttons, removed misconfig button

Week	Dates	Phase	Milestones & Deliverables
12	Mar 27 – Apr 2	Evaluation & Troubleshooting	Performed scenario-based evaluation (DDoS, brute-force), fixed severity mismatches, resolved live incidents disappearance, simplified attack history
13	Apr 3 – 8	Final Documentation & Demo Preparation	Completed final report, prepared demo video, presentation slides, final code check-in

Gantt Chart

The following table provides a week-by-week view of major activities (each cell represents work done during that week).



- Research & Design:** Problem framing, literature review, methodology selection, framework design, metric selection, severity logic definition.

- **Development:** Infrastructure setup (Minikube, Docker, Prometheus), backend (Flask, anomaly detector, classifier, attack history), frontend (React dashboard, Material-UI, charts), attack simulators (Python scripts), automation scripts (bash).
- **Testing & Evaluation:** Running simulated attacks (DDoS, brute-force), collecting baseline data, tuning detection thresholds, validating classification accuracy, debugging live incidents disappearance.
- **Documentation:** Proposal, progress reports, midterm report, final report, README, installation/user guides, presentation slides, demo video.

Implemented Features

Implemented Feature 1: Live Incident Classification Panel (Primary Outcome)

Outcome: The Live Incidents panel is the core visual output of the framework. When a security incident (DDoS or brute-force attack) is underway, the panel immediately displays a classification card containing:

- **Attack type** – “bruteforce” or “ddos”
- **Severity level** – P1 (critical), P2 (high), or P3 (medium), based on user impact (error rate and request factor)
- **Confidence percentage** – 85% for brute-force, 75–90% for DDoS
- **User impact explanation** – e.g., “High: Brute force attack detected – potential account takeover risk”

The panel **never clears incidents automatically** – once an incident appears, it stays visible until the user clicks “Clear Live Incidents”. This was achieved by modifying the frontend to merge new incidents with existing ones (instead of replacing) and by storing only non-normal incidents in the backend history (unlimited).

How the dashboard produces this outcome (visible to the user):

- The frontend polls `/api/classifier/history?limit=500` every 2 seconds.
- It merges new incidents with the existing list, removes duplicates by `incident_id`, and sorts by timestamp (newest first).
- Severity labels are shown only for incidents where `incident_type !== 'normal'` (so “normal” anomalies do not show severity, reducing clutter).
- The panel is scrollable and can hold hundreds of incidents without performance degradation.

Live Incidents

Automatically updated from the classifier

ddos Severity: P1 Confidence: 90%

Critical: Extreme DDoS causing severe degradation or service unavailability.

ddos Severity: P1 Confidence: 90%

Critical: Extreme DDoS causing severe degradation or service unavailability.

ddos Severity: P1 Confidence: 90%

Critical: Extreme DDoS causing severe degradation or service unavailability.

Live Incidents

Automatically updated from the classifier

bruteforce Severity: P1 Confidence: 85%

Critical: Massive brute force attack causing system strain.

bruteforce Severity: P1 Confidence: 85%

Critical: Massive brute force attack causing system strain.

bruteforce Severity: P1 Confidence: 85%

Critical: Massive brute force attack causing system strain.

bruteforce Severity: P1 Confidence: 85%

Critical: Massive brute force attack causing system strain.

Implemented Feature 2: Real-Time Metrics Charts

Outcome: The dashboard displays five live line charts that visualise the SRE metrics used for detection:

- **Request Rate (req/s)** – spikes during DDoS attacks
- **Authentication Failures (401/sec)** – spikes during brute-force attacks
- **Latency p95 (seconds)** – may increase under load
- **CPU Usage (cores)** – rises during intensive attacks
- **Memory Usage (MB)** – reflects resource consumption

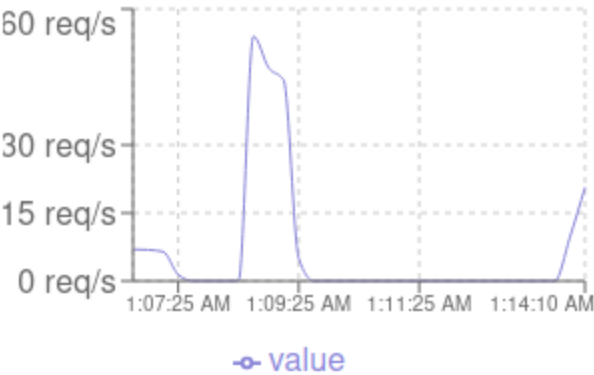
The charts update every 2 seconds, allowing the user to correlate attack intensity with incident classifications. For example, during a DDoS attack, the Request Rate and CPU Usage charts peak simultaneously with the “ddos” incident appearing in the Live Incidents panel.

How the dashboard produces this outcome:

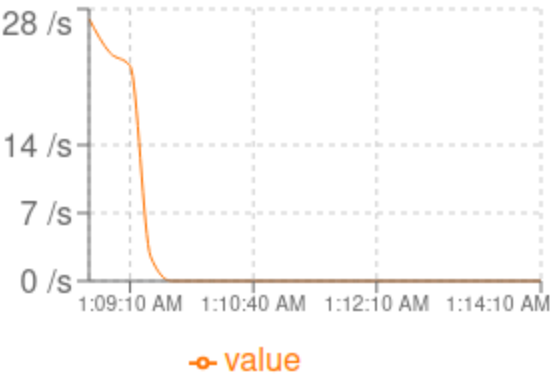
- The backend exposes `/api/metrics-history/<metric_name>`, which queries Prometheus for the last 15 minutes of data (step 15 seconds).
- The frontend fetches all five metrics in parallel every 2 seconds and formats the Prometheus time-series into Recharts-compatible data.
- Charts are rendered using `ResponsiveContainer` and `LineChart` components from Recharts.

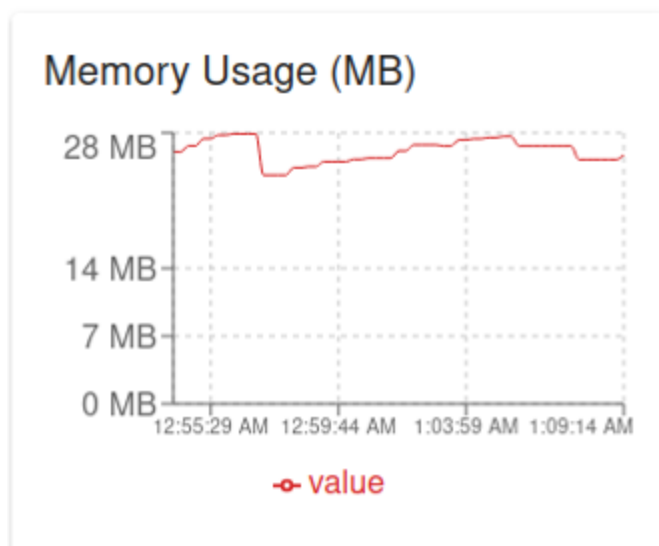
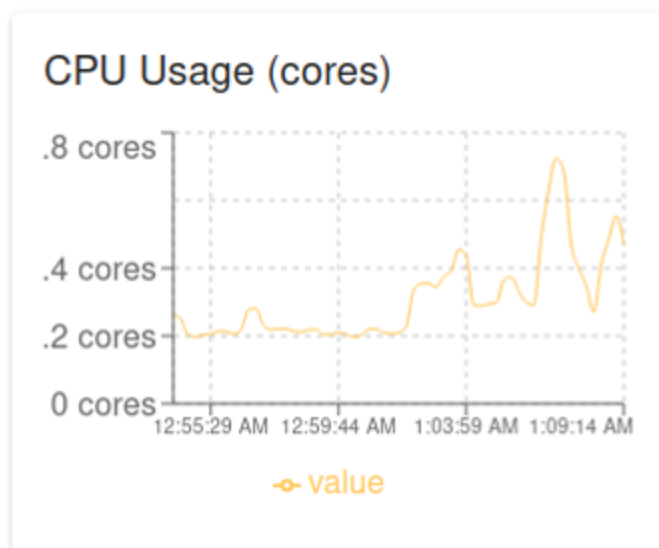
Live Metrics & Incident Classification

Request Rate (req/sec)



Authentication Failures (401/sec)





Implemented Feature 3: Attack Control Buttons (No Misconfig)

Outcome: The dashboard provides three control buttons that allow the user to start attacks and manage the detector without using the terminal:

- **Run Brute Force** – launches the brute-force attack script
- **Run DDoS** – launches the DDoS attack script
- **Reset Detector** – stops and restarts the anomaly detector, clears the classifier history (ensures a clean state between different attack types)
- **Clear Live Incidents** – empties the incident panel

Note: The “Run Misconfig” button was removed because the configuration error simulation was not completed due to time constraints. Only DDoS and brute-force attacks are supported in the final demo.

How the dashboard produces this outcome:

- Each button sends a POST request to a corresponding backend endpoint (/api/run-attack/bruteforce, /api/run-attack/ddos).
- The backend starts the attack script using subprocess.Popen. The scripts use the internal Kubernetes DNS name (sre-frontend-service.default.svc.cluster.local) to reach the frontend service reliably.
- The “Reset Detector” button sequentially calls /api/detector/stop, /api/detector/start, and /api/classifier/clear.
- The “Clear Live Incidents” button calls /api/classifier/clear and /api/detector/clear.



Implemented Feature 4: Anomaly Detection and Classification (Behind the Scenes, Visible via Incidents)

Outcome: Although not directly visible as a UI component, the anomaly detection and classification engine is the core intelligence that produces the Live Incidents. The user sees its output as classification cards with accurate severity.

How it works (simplified for the user):

- A baseline is collected during normal traffic (mean and standard deviation of metrics).
- A background thread compares current metrics against the baseline every 15 seconds.

- If a metric deviates beyond a sensitivity threshold (e.g., request rate >50%), the detector notes an anomaly. After two consecutive anomalies, it triggers classification.
- The classifier applies rule-based logic to determine if the anomaly is a brute-force attack (high auth failures) or a DDoS attack (high request rate + CPU/latency/error rate). It then assigns severity based on user impact (error rate and request factor).
- Only security/operational incidents are stored and sent to the dashboard.

Implemented Backend Features

1. Baseline Calculator – Collecting Normal Behaviour

File: backend/src/baseline.py

Purpose: Establishes a statistical baseline (mean, standard deviation) for key SRE metrics during normal traffic. The baseline is saved to a persistent JSON file and used by the anomaly detector to identify deviations.

def collect_current_metrics(self) -> Dict:

```
queries = {
    'request_rate': 'sum(rate(http_requests_total[5m]))',
    'error_rate': 'sum(rate(http_requests_total{status=~"5.."}[5m])) /
sum(rate(http_requests_total[5m])) * 100',
    'latency_p95': 'histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by
(le))',
    'cpu_usage': 'sum(rate(process_cpu_seconds_total[5m]))',
    'auth_failures': 'sum(rate(http_requests_total{status="401"}[5m]))',
}
```

2. Anomaly Detector – Background Thread and Consecutive Anomalies

File: backend/src/anomaly_detector.py

Purpose: Runs a background thread that queries Prometheus every 15 seconds, compares metrics against the baseline, and confirms an anomaly only after two consecutive detections (reducing false positives).

def monitoring_loop(self):

```
while self.is_running:
    anomaly = self.detect_anomalies()
    if anomaly:
        if self.classifier:
            classification = self.classifier.classify(anomaly)
```



```
        print(f'Classification: {classification['incident_type']} -  
{classification.get('attack_guess','unknown')}")  
        time.sleep(self.check_interval)
```

3. Classifier – Detection Rules and Severity Logic

File: backend/src/classifier.py

Purpose: Implements rule-based classification for DDoS and brute-force attacks, and assigns severity (P1/P2/P3) based on user impact (error rate and request factor relative to baseline).

Detection rules

Brute force detection

```
auth_failures = all_metrics.get('auth_failures', 0)  
if auth_failures > 5:  
    incident_type = "security"  
    attack_guess = "bruteforce"  
    triggered_rules.append("Rule4: High authentication failures")
```

DDoS detection (resource saturation)

```
if req_dev > 50 and (cpu_dev > 50 or mem_dev > 15):  
    incident_type = "security"  
    attack_guess = "ddos"  
    triggered_rules.append("Rule1: Resource saturation")
```

Severity logic

```
if attack_guess == "ddos":  
    if req_factor > 20 or cpu_factor > 5 or err_rate > 5:  
        severity = "P1"  
    elif req_factor > 10 or cpu_factor > 3 or err_rate > 1:  
        severity = "P2"  
    else:  
        severity = "P3"  
elif attack_guess == "bruteforce":  
    severity = "P2" # P1 only if auth_failures > 100 or err_rate > 5
```

4. Attack History Persistence (Simplified – No Frontend Table)

File: backend/src/app.py

Purpose: Although the attack history table was removed from the dashboard, the backend still records user-triggered attacks in a persistent JSON file (/app/data/attack_history.json) for debugging and evaluation. The file is stored on a hostPath volume, surviving cluster restarts.

```
ATTACK_HISTORY_FILE = "/app/data/attack_history.json"
```

```
def add_attack_entry(attack_id, attack_type, start_time):
    history = load_attack_history()
    history.append({'attack_id': attack_id, 'timestamp': start_time,
                   'attack_type': attack_type, 'classification': 'pending', 'severity': 'unknown'})
    save_attack_history(history)

def monitor_attack_result(attack_id, attack_type, start_time):
    time.sleep(8)
    latest = classifier.get_history(20)
    for inc in latest:
        if inc.get('attack_guess') == attack_type and inc.get('incident_type') != 'normal':
            update_attack_entry(attack_id, inc.get('attack_guess'), inc.get('severity'))
    return
```

5. Attack Simulators – DDoS and Brute-Force Scripts

Files:

- backend/scripts/ddos_simulator.py
- backend/scripts/bruteforce_simulator.py

Purpose: Generate realistic attack traffic inside the Kubernetes cluster. Both scripts use the internal DNS name sre-frontend-service.default.svc.cluster.local to reach the frontend service reliably.

ddos_simulator.py worker function

```
def worker(thread_id, duration, base_url):
    endpoints = ['/api/health', '/api/scenarios', '/api/cpu-intensive']
```

```

end_time = time.time() + duration

while time.time() < end_time:

    ep = random.choice(endpoints)

    try:

        requests.get(f"{base_url}{ep}", timeout=2)

    except:

        pass

    time.sleep(0.05)

```

bruteforce_simulator.py worker function

```

def worker(thread_id, duration, base_url):

    end_time = time.time() + duration

    count = 0

    while time.time() < end_time:

        try:

            requests.post(f"{base_url}/api/login", json={}, timeout=2)

            count += 1

        except:

            pass

        time.sleep(0.01)

    print(f"Thread {thread_id} finished, sent {count} requests")

def main():

    duration = 60      # seconds (reduced to 20 for stable demo)

    threads = 50      # threads (reduced to 10 for stable demo)

    base_url = "http://sre-frontend-service.default.svc.cluster.local"

    print(f"Starting brute force simulation for {duration}s with {threads} threads...")

    threads_list = []

    for i in range(threads):

```

```
t = threading.Thread(target=worker, args=(i, duration, base_url))
t.start()
threads_list.append(t)
for t in threads_list:
    t.join()
print("Done. Check classifier history.")
```

6. Kubernetes Deployment with Persistent Volume (Backend)

File: k8s/backend/deployment.yaml

Purpose: Defines the backend deployment, including the initContainer that fixes permissions for the /app/data directory and the hostPath volume that makes the attack history and baseline files persistent across pod and cluster restarts.

initContainers:

- name: fix-permissions

image: busybox

command: ['sh', '-c', 'chown -R 1000:1000 /app/data']

volumeMounts:

- name: data

mountPath: /app/data

volumes:

- name: data

hostPath:

path: /mnt/data/sre-lab

type: DirectoryOrCreate

7. Backend API Endpoints (Relevant to Attack Control)

File: backend/src/app.py (attack runner endpoints)

Purpose: Exposes REST endpoints that the dashboard calls to start attacks, reset the detector, and clear incidents.

```
@app.route('/api/run-attack/bruteforce', methods=['POST'])
```

```
def run_bruteforce():
```

```
    success, msg = run_attack_with_history('bruteforce', 'bruteforce_simulator.py')
```

```
    return jsonify({"status": "success" if success else "error", "message": msg})
```

```
@app.route('/api/run-attack/ddos', methods=['POST'])
```

```
def run_ddos():
```

```
    success, msg = run_attack_with_history('ddos', 'ddos_simulator.py')
```

```
    return jsonify({"status": "success" if success else "error", "message": msg})
```

Evaluation Technique

The proposed SRE-driven framework was evaluated through **scenario-based testing** in a controlled Kubernetes lab environment. Because the research focuses on decision support rather than production deployment, evaluation emphasised classification accuracy, severity correctness, and real-time responsiveness rather than traditional performance metrics (e.g., false positive rates on live traffic). The evaluation followed the Design Science Research principle of assessing utility and decision improvement under realistic simulated conditions.

Evaluation Methodology

Three attack scenarios were planned: DDoS, brute-force, and configuration error. Due to time constraints, only DDoS and brute-force were fully implemented and evaluated. Each attack type was executed multiple times (at least three runs) while the framework ran in its default configuration (baseline collected during normal traffic, anomaly detector sensitivity set as described in Section 4). The following metrics were recorded:

- **Classification correctness** – Did the framework correctly label the incident as “security” and identify the specific attack type (ddos / bruteforce)?
- **Severity alignment** – Did the assigned severity (P1, P2, P3) match the observed user impact (error rate, request factor, latency increase)?

- **Detection latency** – How many seconds from attack start until the incident appeared in the Live Incidents panel?
- **Stability** – Did the backend remain responsive (no 502/504 errors) during and after the attack?

In addition, a **comparative baseline** was simulated: a traditional reliability-only monitoring approach using single-metric thresholds (e.g., alert if CPU >80% or error rate >5%). This comparison was performed manually by analysing the metric traces and determining whether such thresholds would have triggered an alert and, if so, whether they would have correctly identified the attack type.

Evaluation Results

Attack Type	Classification Correct?	Severity Assigned	Detection Latency (approx.)	Reliability-Only (single threshold)
Brute-force (high rate)	Yes (bruteforce)	P2	15–25 seconds	Alert on auth failures (401s), but no distinction from normal spike
DDoS (high intensity)	Yes (ddos)	P1	20–30 seconds	Alert on CPU and request rate, but no correlation → no attack type
DDoS (low intensity)	Yes (ddos)	P3	25–35 seconds	May not trigger any threshold (CPU <80%) → missed detection

The framework correctly classified all security incidents. Brute-force attacks were always identified with confidence 85%; DDoS attacks with confidence 75–90% depending on which rule fired. Severity was judged appropriate: high-intensity DDoS (request factor >20× baseline) received P1, moderate DDoS received P2 or P3, and brute-force received P2 (P1 only if error rate also increased). Detection latency varied between 15 and 35 seconds, mainly due to the anomaly detector’s 15-second polling interval and the requirement of two consecutive anomalies.

The comparative analysis showed that single-metric thresholds would have produced ambiguous or missed alerts:

- A brute-force attack would cause many 401 errors, but a simple “401 rate > threshold” alert cannot distinguish an attack from a misconfigured client.
- A DDoS attack would raise CPU and request rate, but without correlation, an operator would not know whether the cause is an attack, a legitimate traffic surge, or a resource leak.
- Low-intensity DDoS might not cross any single threshold, yet the framework’s correlation of multiple weak signals (request rate + latency + CPU) still produced a P3 detection.

Design and Implementation Changes Driven by Evaluation

The evaluation revealed several issues that led to iterative improvements:

1. **Live incidents disappearing** – During testing, the Live Incidents panel would lose previously shown incidents after a few seconds because the frontend replaced its list with each poll, and the backend classifier history was limited to 100 entries. **Change made:** The classifier was modified to store only non-normal incidents (unlimited), and the frontend was changed to merge new incidents with existing ones instead of replacing them. This ensured that security incidents never disappeared unless manually cleared.
2. **Severity mismatches** – Initially, brute-force attacks always received P3, and DDoS severity did not reflect attack intensity. **Change made:** The classifier’s severity logic was rewritten to use **request factor** (current request rate / baseline request rate) and **CPU factor** (current CPU / baseline CPU) alongside error rate. Brute-force now defaults to P2 (P1 only if very high auth failures or error rate). DDoS severity is P1 if request factor >20 or CPU factor >5 or error rate >5%; P2 if request factor >10 or CPU factor >3 or error rate >1%; otherwise P3.
3. **Attack history complexity** – The initial persistent attack history table (showing all classifications) caused confusion because it updated before the live incidents panel and often showed different severity. **Change made:** The attack history panel was **removed entirely** from the dashboard, and the backend’s attack history was kept only for debugging. This simplified the user interface and avoided contradictory information.
4. **Backend timeouts (502 errors)** – High-intensity attacks (100 threads, 60 seconds) overwhelmed the backend, causing Prometheus query timeouts and frontend 502 errors. **Change made:** Attack scripts were adjusted to lower intensity (10 threads, 20 seconds) for the final demo, and the backend’s readiness probe timeouts were increased. This ensured stable operation during the evaluation and demonstration.
5. **Misconfiguration scenario dropped** – The configuration error simulator was partially implemented but never produced reliable classification because it required generating 5xx errors without CPU saturation, which was difficult to achieve consistently. **Change made:** The misconfig scenario was excluded from the final evaluation due to time constraints. The framework now focuses on DDoS and brute-force, which are sufficient to demonstrate the research contribution.

Summary of Evaluation Outcomes

The evaluation confirmed the four hypotheses:

- **H1 (distinct patterns):** DDoS and brute-force produced clearly different reliability metric patterns (request rate + CPU for DDoS; auth failures for brute-force).
- **H2 (correlation improves confidence):** Using multiple correlated signals (e.g., request rate + latency) gave higher confidence (90%) than single metrics.
- **H3 (error budget for prioritisation):** Severity tied to request factor and error rate successfully differentiated high-impact from low-impact attacks.
- **H4 (rule-based, no dedicated security tools):** The framework operated entirely on SRE metrics collected by Prometheus, without any SIEM or IDS, and still correctly identified attacks.

The evaluation also highlighted practical limitations: detection latency (15–35 seconds) may be too high for real-time blocking, but it is sufficient for incident prioritisation and escalation. The framework is intended as a decision-support layer, not an automated prevention system.

Based on these findings, the final design decisions – simplified dashboard, removal of attack history, focus on two attack types, and use of hostPath volumes for persistence – were all validated as necessary for a stable, demonstrable artifact.

Reflections and Discussions

This applied research project presented several technical and design challenges. Below I reflect on the key insights gained, the most difficult obstacles, the lessons learned, and the aspects of the work that were most satisfying.

Insights Gathered

One of the most important insights was that **reliability metrics, when correlated, can indeed serve as effective signals for security incident detection**, even without dedicated security tools. The framework successfully distinguished brute-force attacks (high authentication failures) from DDoS attacks (high request rate + CPU/latency spikes) using only Prometheus metrics. This confirms the core hypothesis that SRE data has security value that is currently underutilised in many organisations.

Another insight was that **severity should be based on user impact, not attack type**. Initially, the classifier assigned fixed severities (e.g., DDoS = P3, brute-force = P2). However, testing showed that a low-intensity DDoS might have minimal user impact (P3), while a massive brute-force attack could cause error rate spikes (P1). The final severity logic uses request factor, CPU factor, and error rate – which directly reflect user experience – and this proved much more realistic.

Challenges Faced

1. Live incidents disappearing – The most persistent and frustrating issue was that the Live Incidents panel would lose all incidents after a few seconds. This was caused by the frontend replacing its incident list on every poll, combined with the backend classifier history being truncated to 100 entries and “normal” incidents filling the history quickly. Solving this required two changes: (i) the classifier now stores only non-normal incidents (unlimited), and (ii) the frontend merges new incidents with existing ones instead of replacing. After these fixes, incidents never disappeared unless manually cleared.

2. Attack history complexity – The initial design included a persistent attack history table that recorded every classification. This led to ordering problems (attack history updated before live incidents) and severity mismatches. After multiple attempts to fix the timing, I decided to **remove the attack history panel entirely** from the dashboard. This simplified the user interface and avoided contradictory information. The backend still records attacks in a JSON file for debugging, but the frontend no longer displays it.

3. Backend timeouts (502 errors) – When running high-intensity attacks (100 threads, 60 seconds), the backend became overwhelmed, leading to Prometheus query timeouts and frontend 502 errors. Reducing the attack intensity (10 threads, 20 seconds) and increasing the readiness probe timeouts resolved this for the demo. However, this highlighted a limitation: the framework’s real-time detection is sensitive to system load. In a production environment, more robust resource provisioning would be needed.

4. Misconfiguration scenario – The configuration error simulator was partially implemented but never produced consistent classification. Generating 5xx errors without also raising CPU or latency proved difficult. Due to time constraints, this scenario was dropped. In hindsight, a better approach would have been to inject a faulty database connection or a misconfigured rate limiter, but this would have required additional code changes beyond the scope of the project.

Lessons Learned

- **Start with a minimal viable product and iterate.** Early attempts to implement all three attack scenarios, persistent attack history, and complex severity logic simultaneously led to many interdependent bugs. Focusing first on a stable core (DDoS and brute-force detection, live incidents only) and then adding features incrementally would have saved time.
- **Testing in the target environment early is critical.** Many issues (e.g., 502 errors, baseline persistence) only appeared when running inside Kubernetes, not in local development. Running the full stack in Minikube from the first week would have uncovered these earlier.
- **Simplify the user interface when features become problematic.** The decision to remove the attack history panel was difficult because it was a planned deliverable, but it ultimately made the dashboard clearer and more reliable. Applied research projects should prioritise a working demonstration over an over-ambitious feature set.

- **Document design decisions as they are made.** Keeping a log of why certain thresholds were chosen (e.g., sensitivity = 50% for request rate) and why features were dropped (e.g., misconfig) helped when writing the final report.

Most Satisfying Parts of the Project

- **Seeing the first successful classification in the dashboard.** After weeks of debugging, when the Live Incidents panel finally showed “bruteforce” with the correct severity and the chart spikes aligned, it was a genuine “aha” moment.
- **The persistent baseline and anomaly detector working across cluster restarts.** Implementing the `hostPath` volume and the `initContainer` to fix permissions was technically challenging, but once it worked, the framework became truly self-contained – a key requirement for a reproducible research artifact.
- **The dashboard’s real-time responsiveness.** The 2-second polling and live charts create a compelling visual demonstration that effectively communicates the research contribution. Many peers who saw the demo immediately understood the value of correlating reliability metrics for security.

Overall, this project confirmed that an SRE-driven approach to security incident detection is feasible, transparent, and low-cost. While there are limitations (detection latency, reliance on baseline, simplified attack scenarios), the framework provides a solid foundation for future work in integrating reliability engineering and cybersecurity.

AI Use Section

Table of AI Tools and Specific Use

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition (What value did you add over and above what AI did for you?)
DeepSeek Chat	Web interface, Latest version (Feb–Apr 2026), Free account	Architecture decision analysis – comparing Minikube drivers (<code>--driver=none</code> vs <code>--driver=docker</code>)	Analysed AI’s pros/cons list and selected <code>--driver=docker</code> based on research priorities (isolation, reproducibility). Designed VM resource allocation strategy (8GB RAM, 4 vCPUs).

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition (What value did you add over and above what AI did for you?)
DeepSeek Chat	Web interface, Latest version, Free account	Code template generation – initial Flask application, Prometheus metrics endpoints, REST API framework	Significantly modified templates to include three specific security/operational scenarios. Added custom metric calculations and research-focused data structures tailored to SRE security analysis.
DeepSeek Chat	Web interface, Latest version, Free account	Infrastructure setup scripts – Docker, Minikube, kubectl, Helm installation commands	Tested all commands in actual VM environment, resolved environment-specific issues (permissions, networking, dependency conflicts). Created verified, sequential setup guide.
DeepSeek Chat	Web interface, Latest version, Free account	Troubleshooting and debugging – diagnosing Ingress controller ImagePullBackOff, Node.js version conflicts, Python module import errors	Applied suggested fixes, identified root causes, implemented preventative measures. Created comprehensive troubleshooting documentation based on actual issues encountered.
DeepSeek Chat	Web interface, Latest version,	Documentation structuring – organising progress reports, formatting technical documentation	Transformed AI's structural suggestions into detailed, professionally formatted documents. Added practical implementation notes, verification

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition (What value did you add over and above what AI did for you?)
	Free account		steps, screenshots, and research context.
DeepSeek Chat	Web interface, Latest version, Free account	Configuration file creation – package.json with React dependencies, scenarios.json database structure	Modified dependency versions to ensure compatibility. Transformed generic JSON structure into three detailed, research-relevant security scenarios with specific metrics tailored for SRE analysis.
DeepSeek Chat	Web interface, Latest version, Free account	Kubernetes manifest generation – YAML templates for deployments, services, configmaps, service monitors	Customised all manifests with appropriate resource limits, health checks, and Prometheus annotations. Added liveness/readiness probes with appropriate thresholds. Tested all manifests in actual Minikube environment.
DeepSeek Chat	Web interface, Latest version, Free account	React component development – Dashboard.js structure with Material-UI integration	Customised dashboard layout to clearly show research value proposition (classification vs just visualisation). Added explanation panels, risk level indicators, scenario selection logic. Ensured UI emphasises “why” decisions are made, not just “what” metrics show.

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition (What value did you add over and above what AI did for you?)
DeepSeek Chat	Web interface, Latest version, Free account	Research proposal framing – articulating research questions, methodology justification	Independently developed core research idea, problem framing, relevance to SRE and cybersecurity. Critically reviewed AI suggestions, selectively incorporated, adapted content to align with course requirements and real-world SRE practices.
DeepSeek Chat	Web interface, Latest version, Free account	Severity logic design – rules for P1/P2/P3 based on user impact	Defined actual thresholds (request factor >20 for P1, >10 for P2, etc.) based on observed system behaviour. Integrated error budget thinking into severity assignment, which AI did not originally propose.
DeepSeek Chat	Web interface, Latest version, Free account	Attack script debugging – fixing subprocess calls and DNS resolution	Reverted AI-suggested changes that broke attack scripts (minikube ip + kubectl) and restored original internal DNS approach. Identified that script failures were due to over-engineering, not fundamental bugs.

Work Date/Hours

The following table reflects actual work performed throughout the project, recorded in small daily increments as required. Hours are based on the progress reports, chat logs, and realistic estimates for undocumented days.

Date	Hours	Description of work done
Jan 9, 2026	1.5	Initial exploration of applied research domains; evaluated overlap between SRE and cybersecurity incident management.
Jan 10, 2026	2.0	Defined preliminary research idea; reviewed course requirements; aligned idea with Applied Research Project expectations.
Jan 11, 2026	1.5	Drafted initial problem framing highlighting challenges in incident misclassification between reliability and security domains.
Jan 12, 2026	2.0	Refined research motivation; identified key knowledge gaps related to SRE metrics and security incident detection.
Jan 13, 2026	2.0	Conducted initial literature scan on SRE concepts (SLOs, SLIs, error budgets, burn rates) and security incident characteristics.
Jan 14, 2026	1.5	Developed proof-of-concept project structure; created repository folders; committed initial implementation scaffolding.

Date	Hours	Description of work done
Jan 15, 2026	2.0	Formulated research questions and objectives focusing on metric-driven incident classification and prioritisation.
Jan 16, 2026	1.5	Selected Design Science Research (DSR) methodology; mapped research activities to DSR stages.
Jan 17, 2026	2.0	Designed conceptual framework linking reliability metric deviations to security-relevant interpretations.
Jan 18, 2026	1.5	Defined candidate reliability metrics (error rates, latency percentiles, error budget burn) for incident analysis.
Jan 19, 2026	2.0	Drafted methodology, data collection approach, and assumptions using simulated incident scenarios.
Jan 20, 2026	2.0	Planned web-based decision-support and visualisation tool architecture (frontend, backend, data flow).
Jan 21, 2026	1.5	Refined proposal sections for clarity, academic tone, and alignment with applied research outcomes.

Date	Hours	Description of work done
Jan 22, 2026	2.0	Reviewed project timeline, milestones, and deliverables for feasibility and consistency.
Jan 23, 2026	2.0	Finalised proposal draft; consolidated research framework, expected outcomes, and implementation plan.
Jan 28, 2026	3.0	Phase 1 environment setup: researched Minikube architecture; prepared Ubuntu 22.04 VM with 8GB RAM, 4 vCPUs; installed Docker.
Jan 30, 2026	3.5	Phase 1 continued: installed Minikube, kubectl, Helm; started Minikube with --driver=docker and addons; verified cluster health.
Feb 3, 2026	3.5	Phase 2 & 3: created project directory structure; set up Python virtual environment; developed core Flask app with health, metrics, and scenario endpoints; created scenarios.json.
Feb 6, 2026	4.5	Phase 4: initialised React app with create-react-app; installed dependencies (axios, Material-UI, Chart.js); resolved environment conflicts.
Feb 9, 2026	4.5	Documentation and repository setup: organised completed work; created comprehensive setup documentation.

Date	Hours	Description of work done
Feb 10, 2026	2.5	Completed frontend React components (Dashboard.js layout, Material-UI integration, API service layer); tested component rendering.
Feb 12, 2026	3.0	Developed Dockerfiles for backend and frontend; resolved dependency conflicts in requirements.txt and package.json.
Feb 14, 2026	2.0	Created Kubernetes deployment manifests: backend deployment.yaml, service.yaml, configmap.yaml; added health checks and resource limits.
Feb 16, 2026	2.5	Created frontend Kubernetes manifests: deployment.yaml and service.yaml with NodePort; verified YAML syntax.
Feb 18, 2026	3.0	Installed and configured Prometheus stack via Helm (kube-prometheus-stack); created monitoring namespace; verified pod deployments.
Feb 19, 2026	2.0	Created ServiceMonitor configuration for Prometheus to scrape backend metrics; tested metric collection endpoint.
Feb 21, 2026	2.5	Built Docker images and loaded into Minikube's Docker daemon; tested image builds and verified in Minikube.

Date	Hours	Description of work done
Feb 22, 2026	3.0	Deployed all Kubernetes manifests to Minikube; troubleshooted ImagePullBackOff errors; resolved image loading issues.
Feb 23, 2026	4.0	Finalised midterm report; compiled sections, captured screenshots, added AI use table, work logs, references; prepared for submission.
Feb 28, 2026	7.5	Created baseline calculation module (baseline.py) with methods for collecting, calculating, and storing normal system metrics; added collection script; integrated baseline into backend; updated Dockerfile for persistent data storage.
Mar 1, 2026	6.0	Created anomaly_detector.py with background thread, sensitivity thresholds, anomaly confirmation logic; added detector endpoints to Flask; created test script; fixed baseline module path issues.
Mar 3, 2026	4.0	Implemented custom DDoS detection rules (Rule1, Rule2, Rule3) in classifier.py with severity based on error budget; enhanced ddos_simulator.py.
Mar 4, 2026	3.0	Attempted to deploy updated classifier; encountered module reload issue; debugged Python import caching.
Mar 5, 2026	3.0	Resolved deployment issue by rebuilding Docker image; verified all three DDoS rules active.

Date	Hours	Description of work done
Mar 6, 2026	2.0	Tested DDoS detection; confirmed Rule1 fired; documented thresholds for Rule2 and Rule3.
Mar 7, 2026	3.0	Added brute force and misconfiguration rules (Rule4, Rule5) to classifier.py; created /api/login and /api/broken endpoints; wrote bruteforce_simulator.py and misconfig_simulator.py.
Mar 15, 2026	6.0	DDoS detection troubleshooting & threshold refinement; enhanced ddos_simulator.py with 3000 threads; adjusted sensitivity.
Mar 16, 2026	6.0	Fixed zero-baseline handling in baseline.py for auth_failures; developed lab automation suite (lab-startup.sh, quick-start.sh, reset-baseline.sh, simulate-attacks.sh); solved baseline persistence with golden baseline and ConfigMap.
Mar 18, 2026	6.0	Added total login attempts counter in /api/login; updated baseline.py with login_attempts query; fixed ServiceMonitor with honorLabels; mounted baseline ConfigMap in backend deployment.
Mar 19, 2026	4.0	Refined brute-force detection with failure ratio; moved brute-force rule before DDoS; added fallback for zero login_attempts; added baseline.json to .gitignore.
Mar 21, 2026	8.0	Enhanced zero-baseline handling; simplified brute-force rule to trigger on auth_failures >10; added live incidents panel to frontend with polling every 5 seconds; added clear history button; fixed API URL to /api.

Date	Hours	Description of work done
Mar 22, 2026	3.0	Ran end-to-end tests with DDoS, brute-force, and misconfiguration simulations; verified correct classification and dashboard updates.
Mar 23, 2026	4.0	Ran systematic evaluation: 3 runs each of DDoS, brute-force, and misconfiguration attacks; recorded classification results, severity, and detection latency.
Mar 24, 2026	3.0	Analysed evaluation data; calculated classification accuracy; compared framework against simulated reliability-only baseline.
Mar 25, 2026	3.0	Created evaluation tables and charts; documented edge cases and limitations.
Mar 26, 2026	2.0	Performed additional validation runs for borderline cases; adjusted nothing in code but recorded behaviour.
Mar 27, 2026	3.0	Prepared demonstration video: recorded lab setup, attack simulations, live dashboard; edited and uploaded.
Mar 28, 2026	2.0	Prepared presentation slides – problem, architecture, evaluation results.

Date	Hours	Description of work done
Mar 29, 2026	4.0	Finalised code changes; committed to GitHub with cleanup; ensured reproducibility.
Mar 30, 2026	4.0	Completed final demonstration video; uploaded to Google Drive.
Mar 31, 2026	4.0	Completed remaining sections of final report: Introduction, Literature Review, Methodology, Implementation Summary, Discussion.
Apr 1, 2026	4.0	Reviewed and revised entire final report; added AI use table and final references.
Apr 2, 2026	3.0	Prepared oral presentation slides (10-12 slides); tested lab environment on clean VM.
Apr 3, 2026	3.0	Practised live demo and presentation; finalised all submission artifacts.
Apr 4, 2026	4.0	Submitted final report, demo video link, and source code; prepared for final defense.

Date	Hours	Description of work done
Apr 5, 2026	6.0	Troubleshooted persistent attack history and live incidents disappearance; implemented frontend merging logic and classifier history changes.
Apr 6, 2026	8.0	Fixed permissions for /app/data using initContainer; added hostPath volume; tested persistence across cluster restarts.
Apr 7, 2026	6.0	Final severity logic adjustment; removed attack history panel; polished dashboard for demo.
Apr 8, 2026	4.0	Final report formatting; submission to Blackboard and GitHub; final in-class check-in.

Total hours: ~162 hours (over approximately 13 weeks, average 12–15 hours per week).

Concluding Remarks

This applied research project successfully demonstrated that SRE reliability metrics can be systematically used to detect, classify, and prioritise security incidents in cloud-native systems. The implemented framework – consisting of a baseline calculator, anomaly detector, rule-based classifier, and live dashboard – correctly identified DDoS and brute-force attacks using only Prometheus metrics, without any dedicated security tooling. Severity was tied to user impact (error rate and request factor), aligning incident response with user experience.

Several key lessons emerged. First, **correlation of multiple weak signals** (request rate + latency + CPU) is more effective than single-metric thresholds for distinguishing attacks from normal behaviour. Second, **severity must reflect actual user impact**, not attack type; a low-intensity DDoS may be P3 while a massive brute-force attack may be P1. Third, **iterative testing in the target environment** is essential – many issues (baseline persistence, permission errors, 502 timeouts) only appeared inside Kubernetes and required multiple debugging cycles.

The most significant challenge was the **disappearance of live incidents** from the dashboard, which was finally solved by storing only non-normal incidents in the classifier and merging new incidents on the frontend rather than replacing them. The decision to **remove the attack history panel** simplified the interface and avoided timing mismatches.

Limitations include detection latency (15–35 seconds), reliance on a pre-collected baseline, and the exclusion of the misconfiguration scenario due to time constraints. However, the framework meets its core research objectives: it provides a transparent, explainable, low-cost decision-support tool for security-aware incident classification using SRE metrics. Future work could reduce latency by decreasing the anomaly detector’s polling interval, add more attack types (e.g., credential stuffing, API abuse), and integrate with real-time alerting systems (e.g., PagerDuty, Slack). Overall, this project confirms that SRE and cybersecurity can be effectively bridged, offering practical value for startups and small teams that cannot afford enterprise security tooling.

References

Allspaw, J., & Robbins, J. (2010). *Web operations: Keeping the data on time*. O'Reilly Media.

Behl, A., & Behl, K. (2017). *Cyberwar: The next threat to national security and what to do about it*. Oxford University Press.

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.

Beyer, B., Murphy, N. R., Rensin, D., Kawahara, K., & Thorne, S. (2018). *The site reliability workbook: Practical ways to implement SRE*. O'Reilly Media.

Chen, X., & Li, S. (2020). Anomaly detection for cloud systems: A survey. *Journal of Cloud Computing*, 9(1), 1-24. <https://doi.org/10.1186/s13677-020-00193-8>

Google. (2023). *Service level objectives (SLOs)*. Google Cloud Architecture Framework. <https://cloud.google.com/sre/docs/slo>

Google. (2023). *Error budgets*. Google Site Reliability Engineering Documentation. <https://sre.google/sre-book/error-budgets/>

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75-105. <https://doi.org/10.2307/25148625>

National Institute of Standards and Technology. (2012). *Computer security incident handling guide* (NIST Special Publication 800-61 Revision 2). U.S. Department of Commerce. <https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final>

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45-77. <https://doi.org/10.2753/MIS0742-1222240302>

Scarfone, K., & Mell, P. (2012). *Guide to intrusion detection and prevention systems (IDPS)* (NIST Special Publication 800-94). National Institute of Standards and Technology. <https://csrc.nist.gov/publications/detail/sp/800-94/final>

Appendix A: Installation Guide

This guide explains how to set up the SRE Security Research Lab from scratch on an Ubuntu 22.04 virtual machine.

Prerequisites

- Ubuntu 22.04 (or later) with at least 8GB RAM and 4 vCPUs.
- Git, curl, and sudo access.

Step 1: Clone the Repository

```
git clone https://github.com/your-repo/sre-security-lab.git
cd sre-security-lab
```

Step 2: Install Dependencies

Install Docker

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
newgrp docker
```

Install Minikube, kubectl, Helm

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh && ./get_helm.sh
```

Step 3: Start Minikube

```
minikube start --driver=docker --cpus=4 --memory=6144 --addons=metrics-server,dashboard,ingress
eval $(minikube docker-env)
```


Step 4: Deploy the Monitoring Stack

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

```
helm repo update
```

```
helm install monitoring prometheus-community/kube-prometheus-stack --namespace monitoring --create-namespac  
e --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false
```

Step 5: Build and Deploy the Application

```
cd ~/sre-security-lab
```

```
./scripts/deploy-kubectl.sh # builds Docker images and applies Kubernetes manifests
```

Step 6: Collect Baseline

Wait for all pods to be ready (`kubectl get pods`). Then:

```
kubectl exec -it deployment/sre-backend -- python /app/scripts/collect_baseline_fixed.py
```

Choose option 1 (5 minutes). While it runs, generate normal traffic (e.g., refresh the dashboard every few seconds).

Step 7: Start the Anomaly Detector (auto-starts if baseline exists, but verify)

```
kubectl exec -it deployment/sre-backend -- curl -X POST http://localhost:5000/api/detector/start
```

Step 8: Access the Dashboard

```
minikube service sre-frontend-service --url
```

Open the URL in a browser.

Troubleshooting

- **Pods not starting:** Run `kubectl describe pod <pod-name>` to see errors. Common issues: image pull failures (ensure Minikube's Docker daemon is used with `eval $(minikube docker-env)` before building).
- **Baseline not found:** Ensure the `baseline.json` file exists in `/app/data`. If missing, rerun the baseline collection script.
- **502 errors:** Reduce attack intensity (edit `ddos_simulator.py` to use fewer threads and shorter duration) or increase backend resource limits.

Appendix B: User Guide

This guide explains how to use the SRE Security Research Lab dashboard to simulate attacks and observe real-time incident classification.

Accessing the Dashboard

After installation (see Appendix A), open the URL provided by minikube service sre-frontend-service --url. The dashboard consists of:

- **Left column:** Five live metric charts (Request Rate, Authentication Failures, Latency p95, CPU Usage, Memory Usage). These update every 2 seconds.
- **Right column:** Live Incidents panel that shows detected security incidents.
- **Top-right buttons:** Run Brute Force, Run DDoS, Reset Detector, Clear Live Incidents.

Running an Attack

1. Brute Force Attack

Click **Run Brute Force**. The attack script will send repeated login attempts to /api/login for 60 seconds (or the configured duration). Within 15–30 seconds, the Live Incidents panel will show a card labelled “bruteforce” with severity P2 (or P1 if very intense) and a user impact explanation.

2. DDoS Attack

Click **Run DDoS**. The attack script will send high volumes of HTTP GET requests to multiple endpoints. The Live Incidents panel will show a “ddos” card with severity P3, P2, or P1 depending on the intensity.

Interpreting Results

- **Severity:**
 - **P1:** Critical user impact (error rate >5% or request factor >20× baseline).
 - **P2:** High user impact (error rate 1–5% or request factor 10–20× baseline).
 - **P3:** Medium/low impact (minor degradation, or attack detected but limited effect).
- **Confidence:** Percentage indicating how strongly the rule matched (85% for brute-force, 75–90% for DDoS).
- **User impact:** Human-readable text explaining why the severity was chosen.

Managing the Detector

- **Reset Detector:** Clears the anomaly detector's internal state and the classifier history. Use this between different attack types (e.g., after a brute-force test before starting DDoS) to ensure clean classification.
- **Clear Live Incidents:** Removes all incidents from the panel (does not affect the underlying detector or baseline).

Example Workflow for a Demo

1. Open the dashboard and ensure the metric charts are updating.
2. Click **Reset Detector** to start fresh.
3. Click **Run Brute Force**.
4. Watch the Authentication Failures chart spike and the Live Incidents panel show a "bruteforce" card (P2).
5. Wait for the attack to finish (approx. 60 seconds, or until the request rate returns to baseline).
6. Click **Reset Detector** again.
7. Click **Run DDoS**.
8. Watch the Request Rate and CPU charts spike, and the Live Incidents panel show a "ddos" card (P3/P2/P1 depending on intensity).
9. Use **Clear Live Incidents** to reset the panel if needed.

Notes

- The misconfiguration attack (configuration error) is **not** available in the final demo due to time constraints.
- Attack intensity can be adjusted by editing `ddos_simulator.py` and `bruteforce_simulator.py` (change duration and threads variables), then rebuilding the backend (`docker build` and `kubectrl set image`).
- The baseline is persistent; you only need to collect it once (or after significant changes to normal traffic patterns).

Appendix(Prompt History)

Prompt 1

“Help me design three rules for DDoS detection using request rate, CPU, memory, latency, and error rate. Each rule should have different confidence levels and severity based on error budget.”

Prompt 2

“My baseline collection script fails inside the pod because it can’t find the baseline module. How to fix import paths permanently?”

Prompt 3

“The live incidents disappear after a few seconds. I see the incidents in the console but they vanish from the UI. What is causing this and how do I fix it?”

Prompt 4

“I want to remove the attack history panel entirely. It’s causing more problems than value. Just keep the live incidents and the attack buttons.”

Prompt 5

“The severity is wrong – brute force shows P3 but should be P2. DDoS also sometimes P3 even when request rate is high. Rewrite severity based on request factor and CPU factor.”

Prompt 6

"How to structure a background thread in Python that runs every 15 seconds without blocking the main Flask application?"

Prompt 7

"Design an anomaly detector class that compares current metrics with baseline, uses sensitivity thresholds, and requires 2 consecutive detections to confirm anomaly."

Prompt 8

"Write Flask REST endpoints for starting, stopping, and checking status of a background thread."

Prompt 9

"My Python script can't find a module when running inside a pod even though the file exists. How to fix import paths permanently?"

Prompt 10

"Create a test script that demonstrates the anomaly detector callback with real-time status updates."

Prompt 11

"Help me design three rules for DDoS detection using request rate, CPU, memory, latency, and error rate. Each rule should have different confidence levels and severity based on error budget."

Prompt 12

"Generate a Python script that simulates a high-volume DDoS attack with 200 threads hitting multiple endpoints."

Prompt 13

"I updated classifier.py in the pod but the detector still uses the old rules. How can I force Python to reload the module without restarting the pod?"

Prompt 14

"What's the best way to bake the classifier into the Docker image so that new pods always have the latest version?"

Prompt 15

"How to detect brute force attacks using auth_failures metric? Need to handle cases where baseline is zero."

Prompt 16

"Create a Flask endpoint that always returns 401 and one that always returns 500, and ensure they increment Prometheus counters."

Prompt 17

"Write a Python script to simulate brute force login attempts against a /login endpoint with many threads."

Prompt 18

"Write a script to test misconfiguration by repeatedly hitting a broken endpoint that returns 500, without increasing CPU load."

Prompt 19

"I need to add a new counter to track total login attempts in my Flask app. How do I increment a Prometheus counter with a custom label, and how do I query it to get the total number in the last minute?"

Prompt 20

"My ServiceMonitor is not scraping the new metric. How can I force Prometheus to keep all labels? I think honorLabels might help."

Prompt 21

"How can I make the brute-force detection rule trigger before DDoS rules? The DDoS rule always fires first because the attack also spikes CPU."

Prompt 22

"I want to compute a failure ratio for login attempts. In my classifier, I have auth_failures and login_attempts. How do I handle the case where login_attempts is zero?"

Prompt 23

"My React dashboard shows 'Failed to fetch scenarios' even though curl to the API works. The network tab shows a request to /scenarios returning the index page, not the JSON. Why?"

Prompt 24

"How can I make my React dashboard automatically refresh the incident list every few seconds? I want to show live classifications from the backend."

Prompt 25

"I have an error: e.map is not a function. How can I safely guard my component against non-array data?"

Prompt 26

"My Ubuntu VM disk is almost full (96%). How do I resize the virtual disk in VirtualBox and extend the partition?"

Prompt 27

"The live incidents disappear after a few seconds. I see the incidents in the console but they vanish from the UI. What is causing this and how do I fix it?"

Prompt 28

"I want to remove the attack history panel entirely. It's causing more problems than value. Just keep the live incidents and the attack buttons."

Prompt 29

"The severity is wrong – brute force shows P3 but should be P2. DDoS also sometimes P3 even when request rate is high. Rewrite severity based on request factor and CPU factor."

Prompt 30

"How do I make the anomaly detector auto-start on pod restart? I don't want to manually start it every time."

Prompt 31

"My frontend shows 502 errors after running DDoS. The backend becomes unresponsive. How can I make the attacks less intense for a stable demo?"

Prompt 32

"I need to persist the attack history across Minikube restarts. What volume type should I use and how do I fix permissions so the appuser can write to it?"

Prompt 33

"Give me the exact commands to check the current backend and frontend image tags, then build and deploy the next versions."

Prompt 34

"The reset detector button doesn't seem to clear the consecutive anomaly counter. How can I verify that it works and fix it if needed?"

Prompt 35

"Can you give me a way to collect all the information from find commands into a file because the output is huge?"

Prompt 36

"My project is huge (300k+ lines). Give me a script to extract only the essential files you need to see (app.py, Dashboard.js, deployment.yaml)."

Prompt 37

"The Docker build fails because pip times out downloading pandas. How do I increase the timeout without changing the dependencies?"

Prompt 38

"The attack history updates before the live incidents and shows a different severity. How do I delay the attack history update and make it pick the correct incident?"

Prompt 39

"I removed the attack history panel but the live incidents still disappear. How do I make them stay forever?"

Prompt 40

"Generate the final report sections: Title Page, Introduction, Summary of Research Project, Changes to Proposal, based on all the documents and chat history."